

IBEX DIFT DOCUMENTATION

Part of SentinelSoC, RISC-V based System-on-Chip for secure IoT applications

This section extends the 3-stage Ibex core from lowRISC with **Dynamic Information Flow Tracking (DIFT)**. DIFT attaches a 1-bit tag to both data and control flow and enforces security policies that prevent illegal writes or control transfers when operands or addresses are marked as tainted.

The implementation is integrated directly into the Ibex pipeline and supports configurable propagation and checking policies through dedicated CSRs. When a policy violation occurs, the processor raises a DIFT exception and flushes the pipeline, preventing potentially malicious information flow.

Unlike static analysis approaches, DIFT performs tracking at runtime and therefore observes the actual execution path of the program. This allows the processor to detect attacks involving tainted pointers, unauthorized memory accesses, and illegal control-flow transfers while preserving the original Ibex architecture.

What is DIFT?

Dynamic Information Flow Tracking (DIFT) is a hardware security mechanism that associates metadata (tags) with data as it moves through the processor. Tags propagate through computations and memory operations, allowing the system to monitor how information flows during execution.

A tag indicates whether information should be treated as trusted or tainted. During execution, arithmetic, logical, memory, and control-flow operations propagate tags according to programmable policies. The processor then checks whether tainted information is allowed to influence sensitive operations.

Policies determine whether tainted information is allowed to reach sensitive sinks such as:

- Memory addresses
- Store data
- Branch targets
- Jump targets
- Program counter updates

If a policy is violated, the processor raises a DIFT exception and halts the offending operation.

The DIFT implementation in this project uses a lightweight 1-bit tagging scheme and introduces minimal architectural changes while providing configurable runtime security checks.

High-level DIFT flow

Tags are maintained in three locations:

- **Register file:** one 1-bit tag per architectural register.
- **Data memory shadow RAM:** one 1-bit tag per memory word.
- **Program counter tracking:** PC taint information maintained in the fetch stage.

Two CSRs control DIFT behavior:

1. TPR (Tag Propagation Register)

Determines how tags propagate for different instruction classes. Propagation policies are software programmable, allowing different security configurations without modifying hardware.

2. TCR (Tag Control Register)

Enables or disables taint checks for various instruction classes and determines which operations should trigger DIFT exceptions.

The **Tag Management Unit (TMU)** decodes the current instruction together with the TPR and TCR settings and generates:

- Propagation mode (AND/OR/CLEAR/OLD)
- Source tag checks
- Destination tag checks
- Program counter checks

The execution stage computes new tags and detects ALU-related violations.

The LSU performs data and tag accesses in parallel and checks load/store operations.

Any tag violation is propagated to the controller, which treats it as an exception and flushes the pipeline.

This architecture allows tag computation and checking to occur alongside normal instruction execution with relatively small hardware overhead.

Implementation details

Tag propagation modes are defined in `ibex_pkg.sv` as:

- `ALU_MODE_OLD`
- `ALU_MODE_AND`
- `ALU_MODE_OR`
- `ALU_MODE_CLEAR`

These modes determine how operand tags contribute to the tag of the result, enabling different security policies for arithmetic and logical operations.

TCR bit mapping and TPR field layouts are also defined in `ibex_pkg.sv`.

The major DIFT components are:

- **`ibex_dift_tmu.sv`**
Decodes opcode and function fields and generates propagation modes and check enables according to TPR and TCR settings. The TMU acts as the policy engine of the DIFT framework.
- **`ibex_dift_logic.sv`**
Performs tag propagation and detects ALU-stage taint violations.
- **`ibex_dift_mem.sv`**
Implements a 1-bit shadow memory used during simulation. On FPGA, tag storage is implemented using a dedicated tag RAM.
- **PC tagging**
PC taint is tracked in the IF stage and checked whenever control-flow instructions update the program counter. This enables detection of attacks that attempt to redirect execution using tainted branch or jump targets.
- **Exception handling**
Violations from ALU, memory, and control-flow checks are aggregated by the controller and exposed through `dift_exception_o`.

Where DIFT is enabled

DIFT functionality is guarded by the DIFT macro throughout the RTL.

In this repository, the macro is enabled in:

- The testbench
- `rtl/ibex_if_stage.sv`

It may also be enabled globally through a compile-time define to incorporate DIFT functionality across the entire design.

This approach allows the original Ibex core to be synthesized with or without DIFT support, simplifying comparison between the baseline processor and the security-enhanced version.

Files modified/added for DIFT

Core RTL and pipeline integration

- rtl/ibex_core.sv

Top-level wiring, tag interfaces, and exception aggregation.

- rtl/ibex_if_stage.sv

PC tag tracking and branch target tag selection.

- rtl/ibex_id_stage.sv

Tag reads, forwarding logic, operand tag selection, and TPR/TCR interaction.

- rtl/ibex_ex_block.sv

ALU tag propagation and execution-stage taint checks.

- rtl/ibex_load_store_unit.sv

Tag memory accesses and load/store violation detection.

- rtl/ibex_wb_stage.sv

Tag writeback and forwarding support.

- rtl/ibex_controller.sv

Integration of DIFT exceptions into normal processor exception handling.

- rtl/ibex_cs_registers.sv

Implementation of TPR and TCR CSRs.

- rtl/ibex_pkg.sv

Tag propagation modes and policy definitions.

New DIFT modules

- rtl/ibex_dift_logic.sv

Tag propagation and ALU-stage checks.

- rtl/ibex_dift_tmu.sv

TPR/TCR decoding and check generation.

- rtl/ibex_dift_mem.sv

Shadow memory model used during simulation.

- rtl/ibex_register_file_fpga_tag.sv

Tag register file implementation for FPGA deployment.

FPGA wrapper and IO

- wrapper/dift_nexys_top.sv

Nexys A7 top-level wrapper including tag RAM and exception indicators.

- wrapper/seven_seg_driver.sv

Seven-segment display driver used to show exception counts.

Tests and programs

- sim/ibex_core_tb.sv

Exhaustive DIFT verification testbench.

- mem/normal_test.mem

Clean information flow sanity test.

- mem/attack_test.mem

Tainted pointer store example that intentionally triggers a DIFT exception.

- mem/test_program.mem

Program used by the Nexys A7 FPGA design.

Tests performed (simulation)

The testbench in `sim/ibex_core_tb.sv` performs an exhaustive verification of DIFT functionality, including:

- TPR propagation modes for integer, logical, shift, comparison, and jump instructions.
- Load propagation modes and tag writes to shadow memory.
- TCR source and destination checks.
- Branch and jump instruction checks.
- Load/store address and data checks:
 - `LOADSTORE_CHECK_S`
 - `LOADSTORE_CHECK_D`
 - `LOADSTORE_CHECK_DA`
 - `LOADSTORE_CHECK_SA`
- Program counter taint checking for control-flow instructions.
- CSR accesses to TPR and TCR.
- M-extension operations (`mul`, `div`, `rem`) and their propagation behavior.

The testbench generates:

- `dift_tb_exhaustive_results.log`
- Console pass/fail summaries

to provide complete visibility into DIFT functionality and to verify that propagation and checking policies operate correctly across all supported instruction classes.

Synthesis results

Utilization and timing reports were generated using Vivado after implementation.

Target platform

- **Board:** Nexys A7-100T
- **Device:** `xc7a100tcsg324-1`
- **Tool:** Vivado 2025.2

Clock

- Clock: sys_clk
- Target frequency: **50 MHz**
- Clock period: **20 ns**

Timing analysis

Final implementation timing results:

Metric	Value
Worst Negative Slack (WNS)	1.371 ns
Total Negative Slack (TNS)	0 ns
Worst Hold Slack (WHS)	0.083 ns
Hold TNS (THS)	0 ns
Setup failing endpoints	0
Hold failing endpoints	0

The design successfully meets timing at **50 MHz**, with positive setup and hold slack and no failing endpoints after place-and-route.

Resource utilization

Resource	Usage
Slice LUTs	5794
FDREs	2321
Slice Registers	3039
IOBs	36
BRAM	2.5 tiles
DSP48E1	1

(BRAM usage consists of two RAMB36 blocks and one RAMB18 block.)

The additional hardware required for tag storage and policy enforcement introduces relatively modest resource overhead, demonstrating that DIFT can be integrated into the Ibex pipeline without significant area cost.

Post-synthesis verification

Post-synthesis functional verification was performed using the synthesized netlist to ensure that DIFT functionality was preserved after synthesis.

Verification confirmed:

- Correct tag propagation.
- Proper detection of DIFT violations.
- Correct exception generation and pipeline flushing.
- Functional equivalence between RTL and synthesized implementations.

Timing analysis after place-and-route showed positive setup and hold slack, with no failing endpoints, demonstrating that the design meets timing at 50 MHz on the Nexys A7-100T FPGA.

These results show that the DIFT-enhanced Ibex core has both functionality and security properties from RTL through implementation.